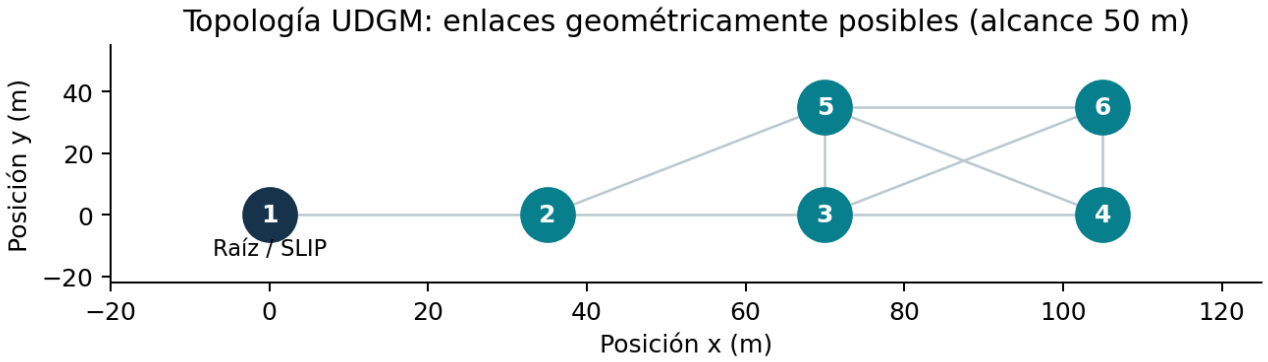
Tarea 4: Mensajería CoAP y MQTT

TEL329 - Redes de Sensores - 2026. Informe de implementación y mediciones. Fecha: 12 de noviembre de 2026.

Se implementaron y ejecutaron las cuatro actividades. CoAP y MQTT se midieron en Contiki-NG/Cooja, y la actividad 4 se verificó con diez lecturas reales del sensor simulado enviadas a Azure IoT Hub y recibidas desde su endpoint integrado. Se conserva además una prueba local independiente. Las temperaturas y humedades son sintéticas, generadas por firmware de Contiki-NG.

1. Escenario experimental

Se utilizó Contiki-NG release/v5.1, Cooja, Java 21 y Linux AArch64 en Docker. Cada escenario usa una raíz RPL Lite y cinco sensores (IDs 2 a 6), IPv6/6LoWPAN y MAC CSMA. La raíz conecta la WSN al anfitrión Linux por SLIP y tunslip6; el anfitrión tiene fd00::1 y ejecuta el servidor CoAP o el broker Mosquitto. Son seis motes, además del anfitrión de servicios.



Fuente: coordenadas de simulations/*.csc. Las líneas muestran alcance geométrico, no padres RPL fijos.

La semilla es 1; cada simulación de comparación dura 240 s, a velocidad 1:1. UDGM utiliza 50 m de alcance, 100 m de interferencia y probabilidad de éxito configurada 1. Esto no elimina contención, colisiones ni retransmisiones. Se esperan 60 s antes de iniciar tráfico, con desfase por ID. La temperatura sintética es 15 + random_rand() % 25: enteros entre 15 y 39 °C. No proviene de un sensor físico.

2. Actividad 1: RPL Lite y CoAP

Los cinco clientes ejecutan coap-sensor.c y notifican un único recurso mediante POST confirmable (CON) a coap://[fd00::1]/temperature. El mensaje JSON contiene temperatura y metadatos node/seq. Se eligió notificación activa mediante POST; no se implementó la extensión Observe ni una consulta GET periódica.

```
coap_init_message(request, COAP_TYPE_CON, COAP_POST, 0);
coap_set_header_uri_path(request, "temperature");
coap_set_header_content_format(request, APPLICATION_JSON);
COAP_BLOCKING_REQUEST(&endpoint, request, response);
```

El servidor aiocoap valida la lectura y devuelve 2.04 Changed con el mismo Message ID y token en el ACK. En el registro de Contiki aparece code=68, codificación binaria de 2.04. La confirmación se observa en la captura y en el callback del sensor. Después de la respuesta, el cliente espera 10 s para enviar otra lectura; por ello el intervalo entre envíos incluye el RTT.

```
Sensor 2, registro real:
COAP_SEND node=2 seq=1 payload={"node":2,"seq":1,"temp":20}
COAP_ACK node=2 seq=1 code=68 rtt_ticks=54

Servidor: {"event":"COAP_RECEIVED","node":2,"seq":1,"temp":20}
```

Se obtuvieron 90 notificaciones y 90 confirmaciones, con 18 intercambios por sensor y ningún intercambio sin respuesta en la captura analizada. El RTT medio fue 124.31 ms; mediana 131.90 ms y percentil 95 192.08 ms. Estos resultados corresponden al escenario y semilla indicados.

Evidencias: results/coap/COOJA.testlog, server.log, node2.pcap a node6.pcap, radio.pcap, tunnel.pcap y responses.csv. La salida de terminal de Cooja finaliza con TEST OK. El código adaptado, Makefile y configuración se incluyen en firmware/.

3. Actividad 2: RPL Lite y MQTT

Los cinco sensores ejecutan mqtt-sensor.c con el motor MQTT de Contiki-NG. Se conectan al broker Mosquitto 2.0.11 en [fd00::1]:1883 y publican exclusivamente en tel329/temperature, usando MQTT 3.1.1, QoS 1 y retain desactivado. Cada cliente tiene un identificador distinto: tel329-node-ID. La conexión TCP se mantiene abierta.

```
mqtt_register(&conn, &sensor_process, client_id, event, 128);
mqtt_connect(&conn, "fd00::1", 1883, 60, MQTT_CLEAN_SESSION_ON);
mqtt_publish(&conn, &mid, topic, (uint8_t *)payload,
             strlen(payload), MQTT_QOS_LEVEL_1, MQTT_RETAIN_OFF);
```

Mosquitto recibe el PUBLISH y devuelve PUBACK con el identificador correspondiente. Un suscriptor independiente, mosquitto_sub, recibe el mismo tema y permite comprobar el comportamiento publicación/suscripción. El PUBACK confirma recepción por el broker; no demuestra por sí solo que la aplicación suscriptora haya procesado el dato.

```
MQTT_CONNECTED node=2
MQTT_SEND node=2 seq=1 mid=3 payload={"node":2,"seq":1,"temp":29}
MQTT_ACK node=2 seq=1 mid=3 rtt_ticks=53
Suscriptor:
tel329/temperature {"node":2,"seq":1,"temp":29}
```

Se registraron 85 publicaciones y 85 PUBACK correlacionados. El RTT medio fue 130.41 ms; mediana 127.34 ms y p95 215.96 ms. Las conexiones se inician tras 60 s; la primera publicación ocurre en una iteración posterior. El establecimiento TCP/CONNECT queda en los PCAP, pero no se suma al RTT PUBLISH-PUBACK.

Evidencias: results/mqtt/COOJA.testlog, broker.log, subscriber.log, nodeN.pcap, radio.pcap, tunnel.pcap y responses.csv. El laboratorio utiliza MQTT sin TLS y un broker anónimo dentro del contenedor. No se usa un broker simulado ni se reemplaza MQTT por UDP.

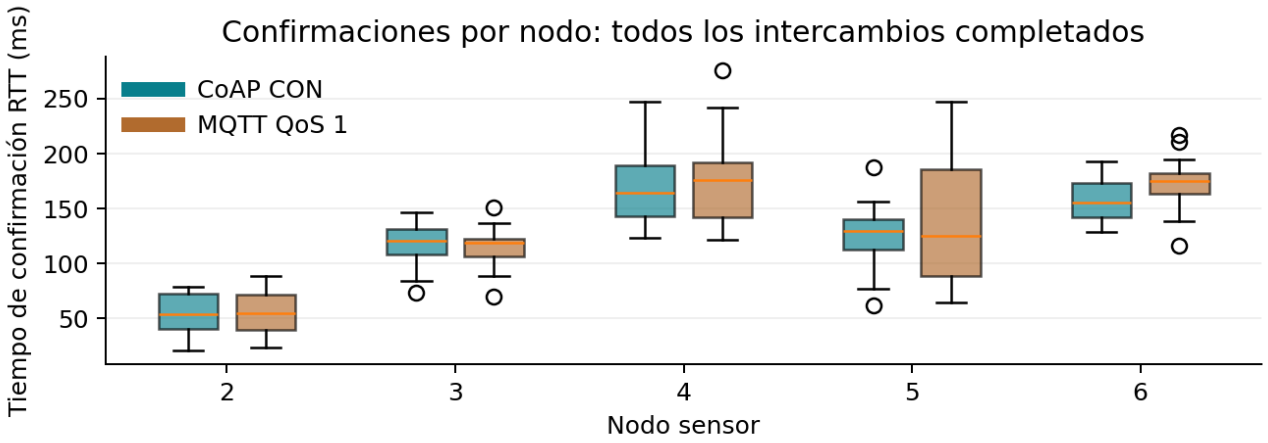
4. Actividad 3: captura y tiempos de respuesta

Se capturaron tres niveles: transmisiones IEEE 802.15.4 mediante el observador de Cooja; paquetes IPv6 en cada sensor mediante un procesador pasivo de NETSTACK; y tráfico del túnel mediante tcpdump. Los bytes se conservan sin alterar. analyze.py exporta las capturas a PCAP y usa tshark para decodificar los protocolos. Los PCAP de sensores y radio tienen timestamps de simulación; el túnel usa el reloj real del contenedor.

El RTT se calcula en el mismo sensor: t(ACK recibido) - t(CON enviado), o t(PUBACK recibido) - t(PUBLISH enviado). CoAP se correlaciona por Message ID y token; MQTT por flujo TCP e identificador de paquete. Se mide desde el primer envío y se conserva el número de solicitudes decodificadas; ese contador no equivale a retransmisiones TCP. Se excluyen paquetes de tránsito RPL con Segments Left mayor que cero. Los registros de aplicación verifican los RTT con una tolerancia de 2 ms; CLOCK_SECOND=1000.

Protocolo	n	Media ms	Mediana ms	p95 ms	Mín-max ms
CoAP CON	90	124.31	131.90	192.08	21.02-247.53
MQTT QoS 1	85	130.41	127.34	215.96	23.30-275.38

Nodo	n CoAP	Media CoAP ms	n MQTT	Media MQTT ms
2	18	53.40	17	55.30
3	18	116.79	17	115.16
4	18	169.39	17	176.18
5	18	124.56	17	132.97
6	18	157.39	17	172.46



Fuente: results/responses.csv; 60-240 s de simulación, semilla 1. Cajas: cuartiles; línea: mediana; puntos: valores atípicos.

5. Muestra trazable y análisis

La tabla presenta las secuencias 1 y 10 de cada nodo en cada protocolo, seleccionadas con una regla fija. Las estadísticas anteriores utilizan todos los intercambios completados, no solo esta muestra. Los CSV conservan los números de trama, instantes, identificadores y RTT para reproducir cada pareja en Wireshark.

Nodo	Secuencia	CoAP RTT ms	MQTT RTT ms
2	1	54.02	53.30
2	10	40.02	63.30
3	1	113.27	150.56
3	10	131.27	116.56
4	1	142.53	140.63
4	10	163.53	141.66
5	1	156.27	87.56
5	10	121.27	64.56
6	1	176.53	187.82
6	10	129.53	194.82

En esta ejecución, la diferencia de medias MQTT menos CoAP fue 6.11 ms. La dispersión por nodo muestra el efecto de los recorridos multihop y de la planificación CSMA. CoAP utiliza UDP y un ACK con respuesta; MQTT agrega TCP, su control de flujo y PUBACK. La captura incluye el procesamiento del servicio, el túnel y las colas, de modo que el tamaño de cabecera por sí solo no explica el resultado.

En los dos escenarios, el registro RPL sitúa al nodo 2 a un salto de la raíz; 3 y 5 a dos saltos; 4 y 6 a tres saltos. La ruta concreta del nodo 6 cambia de padre entre escenarios. Por tanto, el mismo alcance geométrico no garantiza idénticos caminos ni condiciones de contención.

No es una comparación de consultas idénticas: se mide confirmación de telemetría. CoAP 2.04 confirma el procesamiento del POST, mientras PUBACK confirma la aceptación por el broker. Las garantías y los destinatarios son distintos. Los motores de protocolo también consumen números aleatorios y generan tráfico de control diferente. El intervalo CoAP es 10 s después de cada respuesta; MQTT revisa su estado cada 10 s. Estas diferencias limitan la comparación.

Una sola topología y semilla no permiten concluir que un protocolo sea universalmente más rápido. Tampoco equivalen a mediciones sobre hardware físico ni incluyen DTLS/TLS. Para generalizar habría que repetir con varias semillas, distancias, tasas de publicación y pérdidas, manteniendo explícita la métrica. Se incluyen los intercambios no emparejados en unmatched.json, aunque su número en esta ejecución puede ser cero.

6. Actividad 4: integración real con Azure

Se creó el IoT Hub tel329-t4-alopez-20261112 en Brazil South, nivel F1 Free. El portal confirmó USD 0/mes y una cuota de 8.000 mensajes diarios. Se registró contiki-ng-01 con autenticación simétrica. No se contrató un nivel pagado ni se habilitaron complementos de pago.

El escenario utiliza una raíz RPL Lite border router y un sensor HTTP de Contiki-NG (http-sensor.c). El middleware Node.js 22 consulta GET / en la dirección IPv6 del mote, recibe temperatura y humedad y las envía a Azure mediante MQTT/TLS. Un consumidor independiente usa EventHubConsumerClient y AMQP/TLS para leer el endpoint integrado compatible con Event Hubs.

4.a - Resultado del middleware. Se confirmaron diez envíos con event=SENT y mode=azure. El middleware transforma temp/hum en temperature/humidity, conserva node/seq y agrega deviceId, sampledAt y temperatureAlert. Esta última propiedad vale true solo cuando la temperatura supera 30 °C. Las claves permanecen exclusivamente en .env local y no se incluyen en el ZIP.

```
4.b - Respuesta real del mote:
{"node":2,"seq":1,"temp":31,"hum":85}

4.a - Envío real a Azure:
event=SENT, mode=azure, deviceId=contiki-ng-01
node=2, seq=1, temperature=31
humidity=85, temperatureAlert=true

4.c - Consumidor:
event=AZURE_RECEIVED, partition=0
```

4.b - Resultado del sensor. El mote entregó diez respuestas JSON por IPv6/TCP. La humedad se genera en el intervalo 80-89 % y la temperatura en 15-39 °C. Los registros HTTP_SENSOR de Cooja conservan las lecturas originales. No se sustituyó el mote por un generador de datos del anfitrión.

4.c - Resultado del suscriptor. Se recibieron diez eventos únicos desde Azure, con event=AZURE_RECEIVED. check_azure.py compara todas las lecturas originales, los envíos y los eventos recibidos: coinciden identidad, secuencia, temperatura, humedad y alerta. También comprueba que iothub-connection-device-id coincide con contiki-ng-01, autenticado por el servicio.

Evidencias: results/azure/COOJA.testlog, middleware.log, subscriber.log, tunnel.pcap, device-registration.log, readings.csv, summary.json y verification.txt. Las evidencias CoAP, MQTT y del control local se conservan en carpetas separadas. Las cuatro pruebas unitarias del middleware también fueron superadas.

7. Evidencia de Azure y lecturas recibidas

Propiedad del recurso	Valor verificado
IoT Hub	tel329-t4-alopez-20261112
Estado	Active
Nivel	F1 Free (USD 0/mes)
Región	Brazil South
Dispositivo	contiki-ng-01
Lecturas enviadas / recibidas	10 / 10

Se omite la captura del portal en esta edición pública para proteger los datos de la cuenta. La configuración del recurso se comprobó en el portal; los resultados se acreditan mediante los registros originales del mote, middleware y consumidor de Azure.

Secuencia	Temperatura °C	Humedad %	Alerta >30 °C
1	31	85	true
2	33	83	true
3	23	87	false
4	39	82	true
5	19	88	false
6	27	80	false
7	15	81	false
8	29	82	false
9	20	82	false
10	18	89	false

La tabla deriva de los eventos recibidos desde Azure y se contrasta con el mote y el middleware. El suscriptor se inició antes del envío, leyendo desde su instante de arranque. Se utilizó la directiva service para el consumidor; la identidad de dispositivo se configuró mediante el SDK oficial azure-iothub porque la vista Dispositivos del portal devolvía un error. No se ampliaron permisos del usuario ni se deshabilitó TLS.

La prueba local anterior también conservó diez lecturas idénticas entre sensor, archivo y consumidor. Esa prueba se mantiene como control adicional; la evidencia de cumplimiento de la nube es la ejecución real de results/azure/.

8. Reproducción, publicación y referencias

El ZIP contiene firmware, simulaciones, scripts, PCAP, registros originales, datos procesados y este informe en PDF y Markdown. Los servicios y el túnel se ejecutan dentro del mismo contenedor Linux. setup.sh obtiene los commits fijados de Contiki-NG/Cooja y prepara el entorno; run.sh ejecuta los escenarios por separado. No se requieren los archivos de la tarea anterior.

```
bash tarea4/scripts/setup.sh
bash tarea4/scripts/run.sh coap mqtt http
docker run --rm -v "$PWD:/work" tel329-tarea4:1 \
  python3 /work/tarea4/scripts/analyze.py
python3 tarea4/scripts/check_cloud.py
# Con .env de Azure configurado:
bash tarea4/scripts/run_azure.sh
python3 tarea4/scripts/check_azure.py
node --test tarea4/cloud/test.cjs
python3 tarea4/scripts/build_report.py
```

La integración Azure quedó ejecutada y verificada. cloud/LEEME_AZURE.md documenta cómo repetirla con las credenciales locales. El Hub permanece disponible en el nivel F1 gratuito. La adaptación, el informe y las evidencias se publican en el repositorio público https://github.com/aguslopezleg/tel329-tarea4-coap-mqtt, para cumplir el requisito del enunciado.

Conclusión. CoAP y MQTT funcionaron sobre RPL Lite con cinco sensores, respaldados por capturas y tiempos correlacionados. La actividad 4 completó el recorrido sensor Contiki-NG, middleware, Azure IoT Hub y consumidor: diez lecturas fueron verificadas de extremo a extremo. El código y las evidencias se incluyen en el repositorio público de la entrega.

Repositorio público de esta entrega
https://github.com/aguslopezleg/tel329-tarea4-coap-mqtt

Enunciado TEL329 Tarea 4, versión 1.0, 2026
references/TEL329-Tarea4-2026-v1.0.html

Contiki-NG release/v5.1 y fuentes de CoAP/MQTT
https://github.com/contiki-ng/contiki-ng/tree/2b87baf3ebdde3c8e37ca791d2bc84bfd76c49a4

Contiki-NG: Cooja, simulating a border router
https://docs.contiki-ng.org/en/master/doc/tutorials/Cooja-simulating-a-border-router.html

RFC 7252: CoAP, secciones 4 y 5
https://www.rfc-editor.org/rfc/rfc7252

OASIS MQTT 3.1.1, secciones 3.3 y 4.3.2
https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

Microsoft Learn: endpoints de IoT Hub
https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-devguide-endpoints

Microsoft Learn: SDK Event Hubs JavaScript
https://learn.microsoft.com/en-us/javascript/api/overview/azure/event-hubs-readme?view=azure-node-latest